

Single Port RAM

SystemVerilog Functional Verification Report

Name: Vipul Sharma
Employee ID: 6951

1. Introduction

This report documents the functional verification of a Single Port RAM design using SystemVerilog. The verification environment follows a layered testbench architecture consisting of a Transaction, Generator, Driver, Monitor, Scoreboard, Environment, Test, and Testbench top module.

The primary goal of this project is to verify all functional aspects of the RAM including write operations, read operations, reset behavior, simultaneous read/write handling, and functional coverage closure.

2. DUT Specification

The following table summarizes the key specifications of the Design Under Test (DUT):

Parameter	Value
Design Name	Single Port RAM
Data Width	8 bits
Address Width	5 bits
Memory Depth	32 locations (address 0x00 – 0x1F)
Clock Frequency	50 MHz (Period = 20 ns)
Reset Type	Active Low, Asynchronous
Write Operation	Enabled when write_enable=1, read_enable=0
Read Operation	Enabled when read_enable=1, write_enable=0
Simultaneous RW	Not supported; data_out = 8'bz
Reset Behavior	data_out = 8'bz; memory cleared

3. DUT Port Interface

Port Name	Direction	Width	Description
clk	Input	1-bit	Clock signal — 50 MHz
reset	Input	1-bit	Active low asynchronous reset
address	Input	5-bit	Memory address (0–31)
data_in	Input	8-bit	Data to be written
write_enable	Input	1-bit	Enables write operation
read_enable	Input	1-bit	Enables read operation
data_out	Output	8-bit	Data read from memory

4. Testbench Architecture

The verification environment is built using a layered, class-based SystemVerilog testbench. Each component has a specific responsibility and communicates through mailboxes and virtual interfaces.

4.1 Architecture Overview

The testbench consists of the following components:

- Transaction — Data container holding all stimulus and response fields
- Generator — Creates and randomizes transaction objects; sends to Driver via mailbox
- Driver — Receives transactions; drives DUT inputs through virtual interface
- Monitor — Passively observes DUT signals; captures output into transactions
- Scoreboard — Compares actual DUT output against expected (reference model)
- Environment — Instantiates and connects all components; runs the test
- Test — Top-level test class; creates Environment and calls main()
- Testbench — Top-level module; instantiates DUT, Interface, and Test

4.2 Communication Channels

Channel	From	To	Type
gen2driv	Generator	Driver	mailbox #(transaction)
mon2scb	Monitor	Scoreboard	mailbox #(transaction)
dvif	Testbench	Driver	virtual interface
mvif	Testbench	Monitor	virtual interface

4.3 Interface Clocking Blocks

Two clocking blocks are defined inside the interface:

- drv_cb — Driver clocking block; default input #1 output #1; drives all inputs to DUT
- mon_cb — Monitor clocking block; default input #1 output #1; samples all DUT signals

5. Component Details

5.1 Transaction Class

The transaction class holds all randomizable stimulus fields and the captured output:

```
rand bit [7:0] data_in;  
rand bit [4:0] address;  
rand bit      write_enable;  
rand bit      read_enable;  
bit [7:0] data_out; // not randomized
```

Constraints applied:

- `c_r_w_ll` — {`read_enable`, `write_enable`} inside {[0:3]} — allows all combinations
- `c_addr` — address inside {[0:31]} — valid address range only

5.2 Generator

The generator randomizes 500 (you can take any number to repeat) transactions and puts each into the `gen2driv` mailbox. It triggers an 'ended' event upon completion to signal the environment. (I didn't use ended event later and give simple delay by calculation but I will fix this issue)

5.3 Driver

The driver gets transactions from the mailbox, waits for the clocking block edge, and drives all DUT input signals. It also samples functional coverage via `drv_cg` covergroup. After driving, it reads `data_out` back into the transaction.

5.4 Monitor

The monitor passively observes all DUT signals on every clock edge using `mon_cb`. It captures both inputs and outputs into a new transaction and sends it to the scoreboard via `mon2scb` mailbox.

5.5 Scoreboard

The scoreboard maintains a reference memory model (associative array). It compares DUT read output against expected values stored during write operations. Pass/fail counters are maintained as static variables.

- Write: stores `data_in` at address in reference model
- Read: compares `data_out` against reference model — increments pass or fail
- No-op: displays 'No Operation'
- Simultaneous RW: reports unsupported operation

6. Functional Coverage

Functional coverage is implemented inside the Driver class using a covergroup drv_cg:

Coverpoint	Bins	Description
WRITE	wrt[0], wrt[1]	write_enable = 0 and 1
READ	rd[0], rd[1]	read_enable = 0 and 1
DATA_IN	data{[0:255]}	Full 8-bit data range
ADDRESS	address[0]...address[31]	All 32 address locations
WRXRD	cross WRITE×READ (illegal ignored)	All legal RW combinations

The illegal bin (write=1, read=1 simultaneously) is excluded using ignore_bins with binsof + intersect.

Coverage is sampled in the Driver's main() task after each transaction is received from the mailbox.

Final coverage percentage is reported at the end of simulation using:

```
$display("Coverage = %0.2f%%", t1.env.driv.drv_cg.get_coverage());
```

7. Simulation Results

The following results are expected upon successful simulation run:

One correction here is when we are doing \$finish then we can use ended event and when it triggers then it will finish the simulation.

Metric	Expected Value
Total Transactions	500
Scoreboard Pass Count	> 0 (all reads pass)
Scoreboard Fail Count	0
Functional Coverage	100%
WRITE Coverage	100% (both bins hit)
READ Coverage	100% (both bins hit)
ADDRESS Coverage	100% (all 32 bins hit)
WRXRD Cross Coverage	100% (legal bins only)
Simulation Time	~100,000 ns

8. Conclusion

A complete SystemVerilog functional verification environment was designed and implemented for a Single Port RAM DUT. The testbench follows industry-standard layered architecture practices.

The key achievements of this project are:

- Complete layered testbench with Transaction, Generator, Driver, Monitor, Scoreboard, Environment, Test, and Testbench top
- Constrained random stimulus generation with 500 transactions per simulation run
- Functional coverage implemented via covergroup with coverpoints and cross coverage
- Scoreboard-based self-checking verification with pass/fail reporting
- Virtual interface and clocking blocks for clean DUT-TB signal handling
- All major functional scenarios covered including reset, write, read, simultaneous RW, and corner cases

This project demonstrates a solid understanding of SystemVerilog verification methodology and class-based testbench development.

11. Project File Structure

File	Description
design.sv	DUT — Single Port RAM
interface.sv	SV Interface with clocking blocks and modports
transaction.sv	Transaction class with rand variables and constraints
generator.sv	Generator — creates and randomizes transactions
driver.sv	Driver — drives DUT via virtual interface; coverage
monitor.sv	Monitor — observes DUT signals passively
scoreboard.sv	Scoreboard — reference model + pass/fail checking
environment.sv	Environment — connects all components
test.sv	Test class — creates environment and runs test
testbench.sv	Top module — instantiates DUT, interface, test